Studio Nx

Backend(FastAPI/Python) 05 — decode_access_token() · _decode_token()

2026-10-12

Backend(FastAPI/Python) 05 — `decode_access_token()` · `_decode_token()`

항목	내용
파일	`studionx/backend/app/core/security.py`
작업	P1-03 백엔드 공통: 토큰 검사 (PRD 01 FR-105)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	04 `unexpected error handler()` · 06 `get current user()`

1. 하는 일

요청에 실려 온 Access Token이 **이 서버가 만든 진짜 토큰이고 아직 유효한지** 확인한 뒤, 토큰 주인의 `user_id`를 돌려줍니다. 문제가 있으면 401 오류를 던집니다.

`decode_access_token("eyJhbGciOi…")` → `7` (회원 번호)

함수	역할
`decode_access_token()`	Access Token을 검사합니다. 바깥에서 부르는 이름입니다
`_decode_token()`	실제로 검사하는 곳입니다. P1-05의 `decode_refresh_token()`도 같은 함수를 씁니다 (`type`만 다릅니다)

2. 언제 쓰나

로그인이 필요한 **모든 API 요청마다** 한 번씩 불립니다.

순서	일어나는 일
① 브라우저	`GET /api/v1/me/dashboard`를 헤더 `Authorization: Bearer eyJhbGciOi…`와 함께 보냅니다 (JS 02)
② `get_current_user()` (다음 06번)	헤더에서 토큰 문자열을 꺼내 **이 함수**에 넘깁니다
③ 이 함수	검사한 뒤 `7`을 돌려줍니다
④ `get_current_user()`	DB에서 7번 회원을 찾아 라우터에 넘깁니다
실패하면	401 `UNAUTHORIZED`로 응답하고, 브라우저의 `api.js`가 리프레시 쿠키로 새 토큰을 받아 한 번 다시 보냅니다

JWT(토큰)의 생김새

토큰은 점(`.`)으로 나뉜 세 부분으로 되어 있습니다.

부분	예	내용
머리	`eyJhbGciOiJIUzI1NiJ9`	서명 방식 `{"alg": "HS256"}`
내용	`eyJzdWIiOiI3Ii…`	`{"sub": "7", "type": "access", "exp": 1791273600}`
서명	`kQ3x…`	머리와 내용을 **서버 비밀값(`JWT_SECRET`)**으로 계산한 값

칸	뜻
`sub`	토큰의 주인인 회원 번호입니다 (문자열로 저장합니다)
`type`	`access`(30분, API 요청용) 또는 `refresh`(3일, 재발급용)입니다
`exp`	만료 시각입니다 (1970년부터 센 초)

내용은 암호화되지 않습니다. 누구나 풀어서 읽을 수 있으므로 비밀번호 같은 것은 넣지 않습니다. 대신 **서명** 덕분에 내용을 바꿀 수 없습니다. `sub`를 `7`에서 `1`로 바꾸면 서명이 맞지 않게 되고, 비밀값을 모르는 사람은 맞는 서명을 만들 수 없습니다.

3. 코드

import (파일 맨 위)

```
import jwt

from app.core.config import get_settings
from app.core.errors import AppError, ErrorCode
```

`security.py` 맨 위의 import입니다.

import	가져오는 것	쓰는 곳
`jwt`	PyJWT 라이브러리 (`requirements.txt`의 `pyjwt`)	`jwt.decode`, `jwt.PyJWTError`
`get_settings`	`.env`를 읽어 만든 설정 (`app/core/config.py`)	`get_settings().jwt_secret`
`AppError`, `ErrorCode`	01번의 오류 클래스와 코드 (`app/core/errors.py`)	401 `UNAUTHORIZED`

설치할 때의 이름(`pyjwt`)과 import할 때의 이름(`jwt`)이 다릅니다.

함수

```
JWT_ALGORITHM = "HS256"


def decode_access_token(token: str) -> int:
    return _decode_token(token, "access")


def _decode_token(token: str, expected_type: str) -> int:
    invalid = AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    try:
        payload = jwt.decode(
            token,
            get_settings().jwt_secret,
            algorithms=[JWT_ALGORITHM],
            options={"require": ["exp", "sub", "type"]},
        )
    except jwt.PyJWTError:
        raise invalid from None
    if payload["type"] != expected_type:
        raise invalid
    try:
        return int(payload["sub"])
```

```
    except (TypeError, ValueError):
        raise invalid from None
```

문법 풀이

import 문법	뜻
`import 모듈`	모듈 전체를 가져와서 `모듈.이름` 으로 씁니다 (예: `jwt.decode`)
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와서 그대로 씁니다 (예: `Request`)
`app.…`	이 프로젝트(`studionx/backend/app/`)의 파일입니다. `app.core.errors` 는 `app/core/errors.py` 입니다

식	뜻
`jwt.decode(token, 비밀값, algorithms=[…])`	PyJWT 라이브러리의 함수입니다. 서명과 만료(`exp`)를 확인한 뒤 내용을 사전으로 돌려줍니다. 하나라도 틀리면 오류를 냅니다
`options={"require": [...]}`	이 칸들이 없으면 실패합니다
`except jwt.PyJWTError:`	PyJWT의 모든 오류(만료 `ExpiredSignatureError`, 서명 틀림 `InvalidSignatureError`, 형식 틀림 `DecodeError` …)의 부모입니다. 한 번에 잡습니다
`raise invalid from None`	이 프로젝트의 오류로 바꿔 던집니다. `from None` 은 "원래 오류는 이어 붙이지 말라"는 뜻이라서 로그가 짧아집니다
`int(payload["sub"])`	`"7"` 을 `7` 로 바꿉니다. `"admin"` 이면 `ValueError`, 숫자가 아닌 종류면 `TypeError` 가 납니다
`_decode_token` 의 `_`	이 파일 안에서만 쓰는 함수라는 파이썬 관례입니다

토큰	결과
정상 (`sub: "7"`, `type: access`, 만료 전)	`7`
만료됨	401
다른 비밀값으로 서명함	401
내용만 바꿈 (`sub` 7 → 1)	401 (서명 불일치)
`type: refresh`	401
`sub`, `type`, `exp` 중 하나가 없음	401
`sub: "admin"`	401
서명이 없는 토큰 (`alg: none`)	401
`"not-a-token"`, `""`	401

4. 흐름

1. 서명, 만료, 필수 칸을 확인합니다 (`jwt.decode`). 실패하면 401로 응답합니다.
2. `type` 이 `access` 인지 확인합니다. 아니면 401로 응답합니다.
3. `sub` 를 정수로 바꿔서 돌려줍니다. 바꿀 수 없으면 401로 응답합니다.

5. 왜 이렇게 (보안)

선택	이유
`algorithms`를 `HS256` 하나로 고정	토큰 머리의 `alg`는 보내는 사람이 마음대로 쓸 수 있습니다. `"none"`(서명 없음)을 믿으면 누구나 `sub: 1` 토큰을 만들 수 있습니다. 그래서 서버가 정한 방식만 받습니다
`type`을 확인	리프레시 토큰(3일)을 Access Token 자리에 넣어서 30분 제한을 피하지 못하게 합니다
실패 이유와 상관없이 같은 401과 같은 문구	만료인지 위조인지 알려 주면 공격자에게 힌트가 됩니다. 프론트는 어차피 같은 처리(재발급)를 합니다
`"Please sign in again."`의 코드는 `UNAUTHORIZED`	프론트 `api.js`가 재발급을 시도하는 조건과 같은 코드입니다 (JS 02)
검사 부분을 `_decode_token()`으로 분리	P1-05의 `decode_refresh_token()`은 `return _decode_token(token, "refresh")` 한 줄이 됩니다
비밀값은 `.env`의 `JWT_SECRET`에 둠	코드나 Git에 넣지 않습니다. 32바이트 이상이어야 합니다 (짧으면 PyJWT가 경고합니다). 지금 로컬 값은 64바이트이며, 값은 출력하지 않고 길이만 확인했습니다

비밀값이 새어 나가면 누구나 아무 회원의 토큰을 만들 수 있습니다. 그때는 `JWT_SECRET`을 바꾸면 기존 토큰이 모두 무효가 됩니다 (모든 사용자가 다시 로그인해야 합니다).

6. 시험 (`tests/test_security.py` 새 파일, 9개)

토큰 발급 함수(`create_access_token`)는 P1-05에서 만들기 때문에, 시험에서는 `jwt.encode`로 토큰을 직접 만들었습니다.

경우	결과
정상 토큰	`7`
만료, 다른 비밀값, `type: refresh`	401 `UNAUTHORIZED` "Please sign in again."
`sub`, `type`, `exp` 중 하나가 없음 (3개)	401
`sub: "admin"`	401
`"not-a-token"`, 빈 문자열	401
서명이 없는 토큰 (`alg: none`)	401
내용만 바꾼 토큰	401

전체 시험 35개가 통과했고, `ruff` 검사도 통과했습니다.